

DDA5001 Machine Learning

Clustering: k-means

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Recap: PCA

PCA modeling:

$$\mathbf{x}_i = \mathbf{A}\boldsymbol{\theta}_i + \boldsymbol{\mu}$$

Interpretation: Find the **closest** k -dimensional data point $\boldsymbol{\theta}_i = \mathbf{A}^\top \mathbf{x}_i$ to $\mathbf{x}_i$ (projecting $\mathbf{x}_i$ onto the k -dimensional subspace spanned by the columns of $\mathbf{A}$).

Process of applying PCA:

- ▶ Remove the mean: $\mathbf{x}_i = \mathbf{x}_i - \boldsymbol{\mu}$, where $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$.
- ▶ Compute SVD $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$ or eigen decomposition $\mathbf{S} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$.
- ▶ Basis $\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$, principle component $\boldsymbol{\theta}_i = \mathbf{A}^\top \mathbf{x}_i$, and the approximation of $\mathbf{x}_i$ lying in the k -dimensional subspace is $\mathbf{A}\boldsymbol{\theta}_i = \mathbf{A}\mathbf{A}^\top \mathbf{x}_i$.

Extension to low-rank matrix factorization:

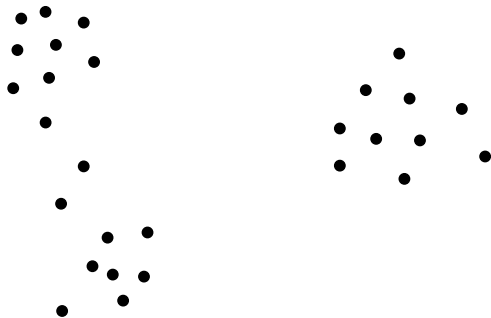
$$\underset{\mathbf{A} \in \mathbb{R}^{d \times k}, \boldsymbol{\Theta} \in \mathbb{R}^{k \times n}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\boldsymbol{\Theta}\|_F^2.$$

One optimal solution to the above (low-rank) matrix factorization problem is given by PCA. And it has infinitely many equivalent optimal solution.

Agenda

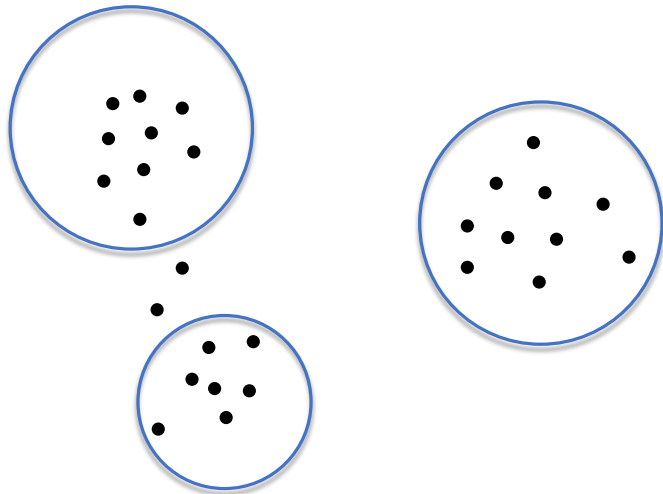
Clustering: k-means

The Unlabeled Samples



- ▶ Unlabeled samples.
- ▶ **Task:** Assign data to several groups / clusters.

Clustering



- This task is very different from classification.

Clustering: Formal Definition

- ▶ Given **unlabeled** data samples $x_1, \dots, x_n \in \mathbb{R}^d$.

Clustering: Assign the unlabeled data to disjoint subsets called **clusters**. The principle is that **points in the same cluster are more similar to each other than points in different clusters**.

Clustering: Formal Definition

- ▶ Given **unlabeled** data samples $x_1, \dots, x_n \in \mathbb{R}^d$.

Clustering: Assign the unlabeled data to disjoint subsets called **clusters**. The principle is that **points in the same cluster are more similar to each other than points in different clusters**.

A clustering algorithm can be represented by a cluster mapping, which is a function

$$C : \{1, \dots, n\} \rightarrow \{1, \dots, k\}$$

where k is the number of clusters.

Intuition for performing clustering:

- ▶ Define certain distance functions.
- ▶ Compute pair-wise distances and put a threshold to assign clusters.

Clustering vs. Classification

- ▶ **Clustering**: Unsupervised, unlabeled data, assign data into clusters without recognizing them. For example, we just cluster dog and cat without recognizing whether x is dog or cat.
- ▶ **Classification**: Supervised, labeled data, recognize the given data, i.e., tell whether x is dog or cat.

▪

k-means Clustering

k-means

One natural method for clustering is **k-means**, which is to cluster by determining a center of each group.

k-means criterion: Choosing C to **minimize**

$$\min_C W(C) = \sum_{j=1}^k \sum_{i:C(i)=j} \|x_i - \mu_j\|_2^2$$

where

$$\mu_j := \frac{1}{n_j} \sum_{i:C(i)=j} x_i \quad n_j = |\{i : C(i) = j\}|$$

k-means

One natural method for clustering is **k-means**, which is to cluster by determining a center of each group.

k-means criterion: Choosing C to **minimize**

$$\min_C W(C) = \sum_{j=1}^k \sum_{i:C(i)=j} \|x_i - \mu_j\|_2^2$$

where

$$\mu_j := \frac{1}{n_j} \sum_{i:C(i)=j} x_i \quad n_j = |\{i : C(i) = j\}|$$

- In k-means, the number of clusters k is assumed to be known as a **prior**.

k-means

One natural method for clustering is **k-means**, which is to cluster by determining a center of each group.

k-means criterion: Choosing C to **minimize**

$$\min_C W(C) = \sum_{j=1}^k \sum_{i:C(i)=j} \|x_i - \mu_j\|_2^2$$

where

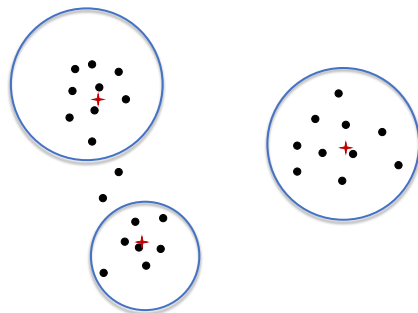
$$\mu_j := \frac{1}{n_j} \sum_{i:C(i)=j} x_i \quad n_j = |\{i : C(i) = j\}|$$

- In k-means, the number of clusters k is assumed to be known as a **prior**.

Interpretation:

- k-means: Find k means of the k clusters.
- Calculate the squared Euclidean distance (ℓ_2 -norm) to measure similarity and dissimilarity.
- Clustering by finding the assignment that provides small within-cluster distance, i.e., minimizing $W(C)$.

k-means: Interpretation



- Intuition: Find k balls (due to Euclidean distance) to enclose each cluster.

How many possible clusters?

How many cluster maps C do we need to consider?

$S(n, k) = \#$ of possible clusterings of n objects into k clusters

$$= \frac{1}{k!} \sum_{j=1}^k (-1)^{k-j} C_k^j j^n$$

where $C_k^j = \frac{k(k-1)\cdots(k-j+1)}{j(j-1)\cdots 1}$.

Examples:

- ▶ $S(10, 4) = 34105$.
- ▶ $S(19, 4) \approx 10^{10}$.

Thus, solving **exactly** a k-means problem is almost impossible.

Minimizing The k-means Criterion

- ▶ No existing (provably) efficient method for solving it.
- ▶ It can be solved exactly (by enumerating) in time $\mathcal{O}(n^{dk+1} \log n)$, which is **exponential time**.
- ▶ This is completely impractical unless both d and k are extremely small.
 - E.g., $d = 2$, $k = 3$ already results in $\mathcal{O}(n^7 \log n)$ solving time.

Minimizing the k-means criterion is a **combinatorial optimization** problem, which is **NP-hard**.

In practice, we resort to **iterative, suboptimal** algorithms—**k-means algorithm**.

Reformulation of k-means

Recall k-means is to solve

$$C^* = \operatorname{argmin}_C \sum_{j=1}^k \sum_{i: C(i)=j} \|x_i - \mu_j\|_2^2.$$

For **fixed** C

$$\mu_j = \operatorname{argmin}_{\mathbf{m}} \sum_{i: C(i)=j} \|x_i - \mathbf{m}\|_2^2.$$

Thus, we can write k-means equivalently

$$C^* = \operatorname{argmin}_{C, \{\mathbf{m}_j\}_{j=1}^k} \sum_{j=1}^k \sum_{i: C(i)=j} \|x_i - \mathbf{m}_j\|_2^2.$$

- ▶ We now have two types of decision variables: 1) The clustering map.
2) The means/centers/**centroids** $\{\mathbf{m}_j\}$.
- ▶ This reformulation gives us an **alternating minimization** framework.

k-means Algorithm

$$C^* = \operatorname{argmin}_{C, \{\mathbf{m}_j\}_{j=1}^k} \left\{ W(C, \{\mathbf{m}_j\}_{j=1}^k) := \sum_{j=1}^k \sum_{i: C(i)=j} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2 \right\}.$$

- ▶ In the formulation above, we have **two** types of decision variables.
- ▶ Recall that optimization algorithm almost always follows the idea: Dividing the original problem into a set of simpler subproblems.
- ▶ Solving the k-means problem for both the clustering map C and the centers $\{\mathbf{m}_j\}$ are hard. **Can we just solve one of them at one time? It could be much simpler than solving the two variables simultaneously.**

k-means Algorithm

$$C^* = \operatorname{argmin}_{C, \{\mathbf{m}_j\}_{j=1}^k} \left\{ W(C, \{\mathbf{m}_j\}_{j=1}^k) := \sum_{j=1}^k \sum_{i: C(i)=j} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2 \right\}.$$

- ▶ In the formulation above, we have **two** types of decision variables.
- ▶ Recall that optimization algorithm almost always follows the idea: Dividing the original problem into a set of simpler subproblems.
- ▶ Solving the k-means problem for both the clustering map C and the centers $\{\mathbf{m}_j\}$ are hard. **Can we just solve one of them at one time? It could be much simpler than solving the two variables simultaneously.**

Alternating minimization algorithm:

- ▶ **Fix** C , update $\{\mathbf{m}_j\}_{j=1}^k$ to minimize $W(C, \{\mathbf{m}_j\}_{j=1}^k)$.
- ▶ **Fix** $\{\mathbf{m}_j\}_{j=1}^k$, update C to minimize $W(C, \{\mathbf{m}_j\}_{j=1}^k)$.

k-means Algorithm

Key advantage: Each subproblem can be solved very easily.

k-means Algorithm

Key advantage: Each subproblem can be solved very easily.

Use t to denote the iteration number.

k-means algorithm

- Fix $C = C^{(t)}$, the update of $\{\mathbf{m}_j\}_{j=1}^k$ is

$$\mathbf{m}_j^{(t+1)} = \frac{1}{n_j} \sum_{i: C^{(t)}(i)=j} \mathbf{x}_i, \quad j = 1, \dots, k.$$

- Fix $\{\mathbf{m}_j\}_{j=1}^k = \{\mathbf{m}_j^{(t+1)}\}_{j=1}^k$, the update of C is

$$C^{(t+1)}(i) = \operatorname{argmin}_{1 \leq j \leq k} \|\mathbf{x}_i - \mathbf{m}_j^{(t+1)}\|_2^2, \quad i = 1, \dots, n.$$

- Iterate the above two steps until convergence.

k-means Algorithm

Key advantage: Each subproblem can be solved very easily.

Use t to denote the iteration number.

k-means algorithm

- Fix $C = C^{(t)}$, the update of $\{\mathbf{m}_j\}_{j=1}^k$ is

$$\mathbf{m}_j^{(t+1)} = \frac{1}{n_j} \sum_{i: C^{(t)}(i)=j} \mathbf{x}_i, \quad j = 1, \dots, k.$$

- Fix $\{\mathbf{m}_j\}_{j=1}^k = \{\mathbf{m}_j^{(t+1)}\}_{j=1}^k$, the update of C is

$$C^{(t+1)}(i) = \underset{1 \leq j \leq k}{\operatorname{argmin}} \|\mathbf{x}_i - \mathbf{m}_j^{(t+1)}\|_2^2, \quad i = 1, \dots, n.$$

- Iterate the above two steps until convergence.

In the update of $C^{(t+1)}(i)$, each data sample $\mathbf{x}_i$ is assigned to exactly one cluster, even though it can be assigned to more than one clusters.

See <https://www.youtube.com/watch?v=5I3Ei69I40s> for an illustration of the process.

k-means Algorithm: Properties

- ▶ Since the Within-Cluster Sum of Squares (WCSS) error is non-increasing and there only exists a finite number of partitioning, the k-means algorithm usually converges after a finite number of iterations. However, it usually converges to a **local optimum**.
- ▶ No guarantee for convergence to global optimum.
- ▶ Thus, **initialization** can be important.

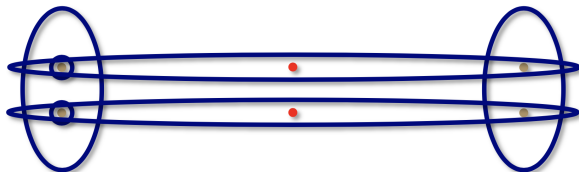
Initialization

The algorithm is typically initialized by setting each m_j to be a **random** point in the dataset, i.e., **random initialization**.

Initialization

The algorithm is typically initialized by setting each m_j to be a **random** point in the dataset, i.e., **random initialization**.

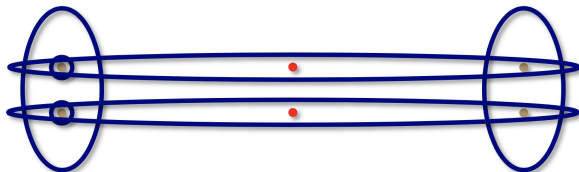
However, depending on the initialization, the algorithm can get stuck in a local minimum. Example: if the two points on the left are chosen as the initial cluster centers ($k = 2$), the k-means algorithm will get stuck, such that **the cluster centers will get stuck at the two red points in the middle**.



Initialization

The algorithm is typically initialized by setting each m_j to be a **random** point in the dataset, i.e., **random initialization**.

However, depending on the initialization, the algorithm can get stuck in a local minimum. Example: if the two points on the left are chosen as the initial cluster centers ($k = 2$), the k-means algorithm will get stuck, such that **the cluster centers will get stuck at the two red points in the middle**.

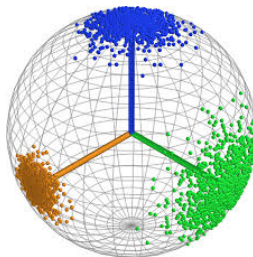


One can avoid this by:

- ▶ Repeating for several random initializations (very common choice in practice).
- ▶ Initialize by **sequentially selecting the cluster centers** as random points in the dataset. When selecting each cluster center, we choose a point with larger probability if this point is further away from the already selected m_j 's. This technique leads to the **k-means ++**.

Remarks

- ▶ k-means algorithm was originally developed at Bell Labs as an approach to **vector quantization** in signal processing.
- ▶ If we replace the squared ℓ_2 -norm with the ℓ_1 -norm distance in the definition of $W(C)$:
 - The center of each region is actually calculated via the **median**.
 - Results in **k-medians clustering**, which can be more **robust**.
- ▶ If we change the Euclidean squared ℓ_2 -norm distance to the **cosine similarity**, we get the **spherical k-means**.



Model Selection for k-means

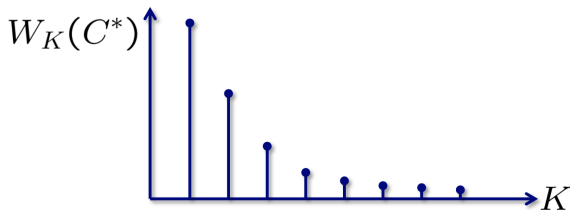
What is model selection used for?

Model Selection for k-means

What is model selection used for? Selecting hyper-parameters.

Sometimes in practice, k is **not** known as a prior, then it becomes a **hyper-parameter**.

Let $W_k(C^*)$ be the optimal loss based on the number of clusters k .



Suppose the “correct” number of clusters is k^* , we expect

- ▶ for $k < k^*$, $|W_k(C^*) - W_{k-1}(C^*)|$ will be **large**.
- ▶ for $k > k^*$, $|W_k(C^*) - W_{k-1}(C^*)|$ will be **small**.

This suggests choosing k to be near the “knee” or “elbow” of the curve.

Matrix Factorization Perspective for k-means

k-means can also be viewed as a special matrix factorization:

$$\mathbf{X} \approx \mathbf{BC},$$

where every column of $\mathbf{B}$ is one of the cluster centers:

$\mathbf{B} = [\mathbf{m}_1, \dots, \mathbf{m}_k]$, and $\mathbf{C}$ has exactly one “1” per column (0 otherwise).

Therefore, the i^{th} column of $\mathbf{X}$ is equal to $\mathbf{B} \times$ the i^{th} column of $\mathbf{C}$.

- Interpretation: Assign each sample (each column of $\mathbf{X}$) to one cluster, represented by the “1” element in the corresponding column in $\mathbf{C}$. In other words, we use a cluster center to approximate each data point in $\mathbf{X}$.
- Learning problem:

$$\min_{\mathbf{B}, \mathbf{C}} \|\mathbf{X} - \mathbf{BC}\|_F^2, \quad \text{subject to} \quad \mathbf{C} \text{ has exactly one “1” per column.}$$

↪ Next lectures: Advanced topics including neural networks and modern training algorithms.